

# Software Architecture & Design Specification

## Traffic ManagementSystem(Simulation)

### A. Software Architecture

#### 1. Architecture Overview

---

TheTraffic Management System (Simulation) is a standalone, single-process application implemented in C/C++ that simulates basic traffic flow at one or more signal-controlled lanes. It is not connected to any network, external service, or real-world traffic hardware — as stated in the SRS (§1.2, §2.1), it exists purely to model vehicle movement through queues and signals for demonstration and analysis.

At its core, the system's architecture is organised around one central idea taken directly from the SRS: queues are the primary data structure representing vehicles waiting in a lane (SRS §1.3, §2.4). Every other capability the SRS asks for — adding vehicles, moving them, detecting congestion, and computing statistics — is built as a thin layer of logic around that queue state, rather than around any other central structure.

The major components interact in a request/response style driven by a single coordinating component. The user interacts only with the Simulation UI, which forwards commands (such as adding a vehicle) to the Simulation Controller. The Simulation Controller is the mediator: it does not itself hold traffic-domain logic, but delegates to the Vehicle Manager (vehicle creation and lifecycle), the Signal Controller (Red/Yellow/Green state per lane), the Congestion Detector, and the Statistics Manager, and relays their results back to the Simulation UI for display. This keeps any single component small and focused, which matters for a data-structures-driven course project where the Queue Manager itself is meant to be the centrepiece.

Concretely: a vehicle is added when the Simulation UI passes a lane and vehicle identifier to the Simulation Controller, which (after input validation) asks the Vehicle Manager to create the vehicle and enqueue it via the Queue Manager — one FIFO queue per lane. Signals are handled by the Signal Controller, which transitions each lane's state through Red, Yellow, and Green on a defined cycle; when a lane turns Green, the Simulation Controller drives the Vehicle Manager to dequeue and “move” the front vehicle out of that lane's queue. Congestion is handled by the Congestion Detector, which compares a lane's current queue length (read from the Queue Manager) against a predefined threshold, exactly as defined in the SRS's definition of congestion (§1.3). Statistics are handled by the Statistics Manager, which listens for vehicle-added and vehicle-departed events and maintains running figures for waiting time, queue length, and vehicles processed (FR8), which the Simulation UI then displays back to the user (FR6).

Because the system is a standalone C/C++ console simulation with no persistence beyond a single run, no network exposure, and no multi-user concerns, the architecture deliberately stays simple: a single process, single thread of control driven by the Simulation Controller, and one FIFO queue per lane as the one piece of state everything else is derived from.

#### 2. Architecture Pattern

This document adopts a Layered Architecture for the system. This is an architectural design decision made for this document and this deliverable — the SRS itself does not prescribe an architecture or a pattern; it only defines required behaviour (§3) and constraints (§2.4). The layering below is one reasonable way to satisfy those requirements, not something the SRS specifies.

The system is organised into four layers:

---

- Presentation Layer — the Simulation UI, which is the only component the user directly interacts with.
- Simulation / Control Layer — the Simulation Controller, which coordinates every request and drives the overall run.
- Traffic Management Layer — the Vehicle Manager, Signal Controller, Congestion Detector, and Statistics Manager, which implement the SRS's traffic-domain behaviour (FR1, FR3, FR4, FR7, FR8).
- Data Structure Layer — the Queue Manager, which owns the per-lane FIFO queues that the SRS identifies as the system's primary data structure (§1.3, §2.4).

A layered pattern was chosen, rather than a single monolithic module, for the following reasons:

- Traceability to the SRS: each layer maps cleanly onto a group of related functional requirements (§2.2's main functions), which makes the traceability matrix in §5 straightforward and defensible rather than forced.
- Isolation of the core data structure: because queues are explicitly called out as the primary data structure (SRS §2.4), keeping the Queue Manager as its own layer means every other component — vehicle logic, signal logic, congestion logic, statistics — depends on queue behaviour through a narrow, well-defined interface instead of manipulating queues directly. This is also what makes the SR2 (queue data protection) security requirement enforceable in one place (see §6).
- Testability: each layer can be exercised and unit-tested independently — for example, the Queue Manager's enqueue/dequeue/length behaviour can be tested without a UI, a signal cycle, or statistics logic running at all.
- A one-directional dependency rule (Presentation → Control → Traffic Management → Data Structure, with results flowing back up) keeps the design easy to reason about for a small team project, at the cost of a small amount of indirection through the Simulation Controller — a trade-off considered acceptable given the project's scope.

### 3. Component Diagram

---

Figure 1 shows the UML component diagram for the system, arranged within the four layers introduced in §2. Solid arrows show the primary request/command flow (top-down); dashed gold arrows show the corresponding result/data flow back up to the Simulation Controller and Simulation UI for display, which realises FR6 (display information).

## Traffic Management System (Simulation) — UML Component Diagram

(Layered Architecture: Presentation → Simulation/Control → Traffic Management → Data Structure)

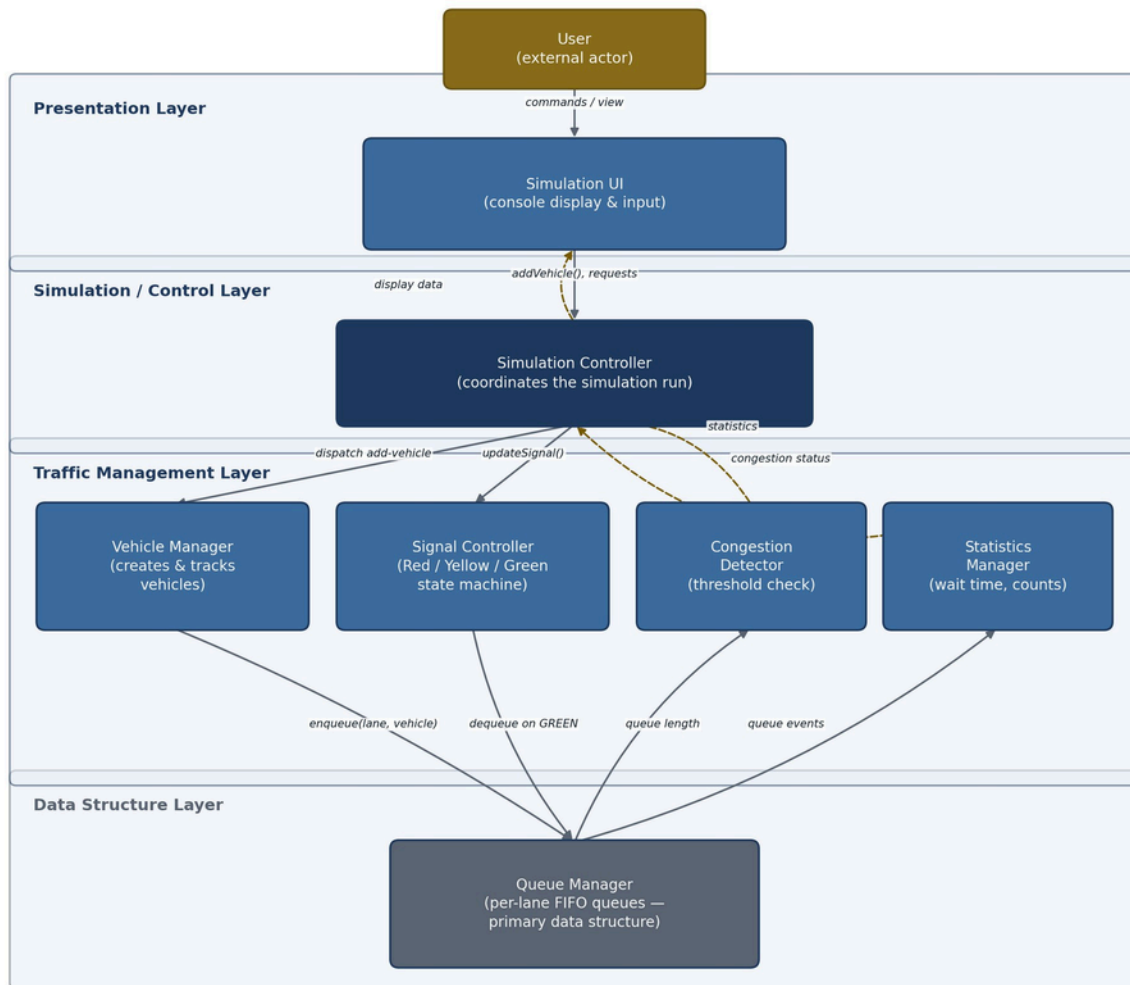


Figure1—UMLcomponentdiagram of the Traffic Management System (Simulation), organised by layer.

## 4. Component Descriptions

Each component below corresponds to one box in Figure 1. “Requirements satisfied” references the FR/NFR IDs from the SRS (§3.1, §3.2).

### Simulation UI

|                         |                                                                                                                 |
|-------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>          | Serves as the system's presentation layer: the single point of interaction between the user and the simulation. |
| <b>Responsibilities</b> | Accept user commands (e.g. add a vehicle); forward requests to the Simulation Controller; render traffic        |

|                               |                                                                                                                                                                                                        |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                               | signal state, vehicle queues, congestion flags, and statistics returned by the Simulation Controller; display confirmation and error messages.                                                         |
| <b>Inputs</b>                 | User commands entered at the console; display data (signal states, queue lengths, congestion status, statistics) returned by the Simulation Controller; validation results (valid / invalid + reason). |
| <b>Outputs</b>                | Rendered console output for the user; outgoing requests to the Simulation Controller.                                                                                                                  |
| <b>Interacts with</b>         | User (actor); Simulation Controller.                                                                                                                                                                   |
| <b>Requirements satisfied</b> | FR6 (display traffic information); NFR2 (usability). Also the entry point for FR1 (add vehicle) and SO2/SR1 (input validation, see §6).                                                                |

### Simulation Controller

|                               |                                                                                                                                                                                                                                                                                               |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>                | Coordinates the simulation run; the single mediator between the Presentation layer and the Traffic Management layer.                                                                                                                                                                          |
| <b>Responsibilities</b>       | Route add-vehicle requests to the Vehicle Manager; drive the Signal Controller's update cycle; on a signal change, direct the Vehicle Manager to move the affected lane's front vehicle; query the Congestion Detector and Statistics Manager and forward their results to the Simulation UI. |
| <b>Inputs</b>                 | Requests from the Simulation UI; signalChanged notifications from the Signal Controller; return values from Vehicle Manager, Congestion Detector, and Statistics Manager.                                                                                                                     |
| <b>Outputs</b>                | Commands to the Vehicle Manager and Signal Controller; queries to the Congestion Detector and Statistics Manager; display data to the Simulation UI.                                                                                                                                          |
| <b>Interacts with</b>         | Simulation UI, Vehicle Manager, Signal Controller, Congestion Detector, Statistics Manager.                                                                                                                                                                                                   |
| <b>Requirements satisfied</b> | Coordinates FR1, FR3, FR4, FR5 end-to-end; NFR1 (performance of the overall run loop).                                                                                                                                                                                                        |

### Vehicle Manager

|                         |                                                                                                                                                                                                                                                                                                                   |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>          | Creates and tracks vehicles within the simulation, and drives their movement out of a lane when instructed.                                                                                                                                                                                                       |
| <b>Responsibilities</b> | Create a vehicle record for a validated add-vehicle request; hand the vehicle to the Queue Manager for enqueueing in the correct lane; on a move instruction from the Simulation Controller, dequeue the lane's front vehicle via the Queue Manager and notify the Statistics Manager of the resulting wait time. |
| <b>Inputs</b>           | addVehicle(laneId, vehicleId) from the Simulation Controller (post-validation); moveVehicle(vehicle) instructions during signal processing.                                                                                                                                                                       |

|                               |                                                                                                                                                                            |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Outputs</b>                | enqueue / dequeue calls to the Queue Manager; recordVehicleAdded / recordWaitTime notifications to the Statistics Manager; confirmation back to the Simulation Controller. |
| <b>Interacts with</b>         | Simulation Controller, Queue Manager, Statistics Manager.                                                                                                                  |
| <b>Requirements satisfied</b> | FR1 (add vehicle), FR5 (vehicle entry/exit), contributes to NFR3 (reliability of queue state).                                                                             |

### Queue Manager

|                               |                                                                                                                                                                                                                                           |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>                | Maintains vehicle queues for individual traffic lanes and handles vehicle insertion/removal and queue-length calculation.                                                                                                                 |
| <b>Responsibilities</b>       | Own one FIFO queue per lane; support enqueue, dequeue, peek-front, and length queries; reject/guard invalid operations (e.g. dequeue on an empty queue) so queue state can never be corrupted by a caller.                                |
| <b>Inputs</b>                 | enqueue(laneId, vehicle), dequeue(laneId), peekFront(laneId), getQueueLength(laneId) calls.                                                                                                                                               |
| <b>Outputs</b>                | Updated per-lane queue state; queue length; the dequeued/peeked vehicle.                                                                                                                                                                  |
| <b>Interacts with</b>         | Vehicle Manager, Signal Controller (drives movement timing), Congestion Detector, Statistics Manager.                                                                                                                                     |
| <b>Requirements satisfied</b> | FR2 (maintain queues), FR5 (entry/exit), FR7 (supplies queue length for congestion detection), NFR1 (performance), NFR3 (reliability), SR2 (queue data protection). The SRS identifies queues as the primary data structure (§1.3, §2.4). |

### Signal Controller

|                               |                                                                                                                                                             |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>                | Maintains and transitions the Red / Yellow / Green signal state for each lane according to a defined cycle.                                                 |
| <b>Responsibilities</b>       | Hold the current signal state per lane; advance state on each updateSignal() cycle tick; notify the Simulation Controller whenever a lane's signal changes. |
| <b>Inputs</b>                 | updateSignal() calls (cycle tick) from the Simulation Controller.                                                                                           |
| <b>Outputs</b>                | getSignalState(laneId) query result; signalChanged(laneId, state) notification.                                                                             |
| <b>Interacts with</b>         | Simulation Controller (directly); Queue Manager (indirectly, since a Green signal is what permits a dequeue).                                               |
| <b>Requirements satisfied</b> | FR3 (signal transitions), FR4 (signal states).                                                                                                              |

### Congestion Detector

|                |                                                                                                            |
|----------------|------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b> | Identifies lanes experiencing congestion, defined by the SRS as a lane having more vehicles than a defined |
|----------------|------------------------------------------------------------------------------------------------------------|

|                               |                                                                                                                                                 |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
|                               | threshold (§1.3).                                                                                                                               |
| <b>Responsibilities</b>       | Query the Queue Manager for a lane's current queue length; compare it against the configured congestion threshold; report the resulting status. |
| <b>Inputs</b>                 | Queue length from the Queue Manager; the configured congestion threshold (SRS §2.5).                                                            |
| <b>Outputs</b>                | detectCongestion(laneId) result — congested / not congested — returned to the Simulation Controller.                                            |
| <b>Interacts with</b>         | Queue Manager, Simulation Controller.                                                                                                           |
| <b>Requirements satisfied</b> | FR7 (detect congestion).                                                                                                                        |

### Statistics Manager

|                               |                                                                                                                                                                          |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>                | Calculates basic traffic statistics: waiting time, queue length, and number of vehicles processed.                                                                       |
| <b>Responsibilities</b>       | Record vehicle-added and vehicle-departed (wait-time) events reported by the Vehicle Manager; maintain running totals/averages; produce a statistics summary on request. |
| <b>Inputs</b>                 | recordVehicleAdded(laneId), recordWaitTime(vehicle) notifications from the Vehicle Manager; queue-length samples from the Queue Manager.                                 |
| <b>Outputs</b>                | calculateStatistics() result (average wait time, queue length, vehicles processed) returned to the Simulation Controller for display.                                    |
| <b>Interacts with</b>         | Vehicle Manager, Queue Manager, Simulation Controller.                                                                                                                   |
| <b>Requirements satisfied</b> | FR8 (calculate statistics).                                                                                                                                              |

## 5. Requirement Traceability Matrix

These requirements are directly defined in the SRS (§3.1, §3.2, §5).

| Requirement | Description              | Component(s)                          |
|-------------|--------------------------|---------------------------------------|
| FR1         | Add vehicle              | Vehicle Manager                       |
| FR2         | Maintain queues          | Queue Manager                         |
| FR3         | Signal transitions       | Signal Controller                     |
| FR4         | Signal states            | Signal Controller                     |
| FR5         | Vehicle entry/exit       | Vehicle Manager + Queue Manager       |
| FR6         | Display information      | Simulation UI                         |
| FR7         | Detect congestion        | Congestion Detector                   |
| FR8         | Calculate statistics     | Statistics Manager                    |
| NFR1        | Performance<br>Usability | Simulation Controller + Queue Manager |
| NFR2        | Reliability              | Simulation UI                         |
| NFR3        |                          | Queue Manager                         |

The security objectives and requirements (§6) are traced separately below, since they cut across the input path rather than mapping to a single functional component:

| Requirement | Description           | Component(s)                                                                |
|-------------|-----------------------|-----------------------------------------------------------------------------|
| SO1         | Data integrity        | Queue Manager (enforcement); all Traffic Management components (compliance) |
| SO2         | Input validation      | Simulation UI / Input Validation step (see §6)                              |
| SR1         | Input validation      | Simulation UI / Input Validation step                                       |
| SR2         | Queue data protection | Queue Manager                                                               |

## 6. Security Architecture

These security objectives (SO1 — Data Integrity, SO2 — Input Validation) and security requirements (SR1 — Input Validation, SR2 — Queue Data Protection) were defined by a teammate at the requirements level. This section shows where each is implemented architecturally, so they are enforced by the system's structure rather than left to individual components to remember to do correctly.

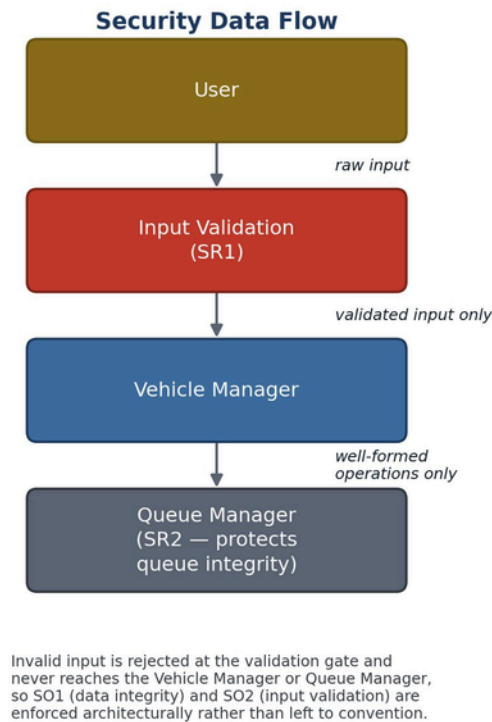


Figure 2 — Security data flow: invalid input is rejected before it can reach vehicle or queue state.

As Figure 2 shows, every user-supplied value passes through an Input Validation step before it reaches the Vehicle Manager. In this architecture, that step is implemented as validation logic invoked by the Simulation UI (or, equivalently, as the first action the Simulation Controller takes on any incoming request) — it is not modelled as a full standalone layer, since its job is a single gate check, not ongoing state management. This directly implements SR1: vehicle and lane inputs are validated before being added or processed, and satisfies SO2 by ensuring invalid data never reaches simulation state.

The Queue Manager independently implements SR2 by never exposing its internal queue storage directly: all access happens through its enqueue / dequeue / length interface (§4, §9), which is defined to reject operations that would corrupt state — for example, a dequeue on an empty queue is refused rather than attempted, and enqueue/dequeue are the only two operations that can change queue contents, so a vehicle cannot be silently duplicated or lost. Because the Vehicle Manager itself only ever receives already-validated input (per SR1), and the Queue Manager only ever accepts well-formed operations through its own interface (per SR2), the two requirements compose to satisfy SO1: vehicle, queue, and signal data stays consistent throughout a run without either component needing to re-check what the other already guarantees.

No additional security architecture (authentication, encryption, network hardening) is included, since the SRS defines the system as a standalone, single-user, offline simulation with no network exposure (§2.1, §2.4) — those concerns are out of scope for this deliverable.

## B. Software Design

### 7. Sequence Diagram — Add Vehicle

Figure 3 shows the Add Vehicle flow. The Simulation UI forwards the request to the Input Validator before anything else happens (implementing SR1/SO2 from §6). The alt fragment shows the two possible outcomes: on the [valid] path, the request proceeds through the Vehicle Manager and Queue Manager, the Statistics Manager is updated, and the Simulation UI confirms success to the user; on the [invalid] path, the Simulation UI immediately reports the error and no vehicle- or queue-manager state is touched.

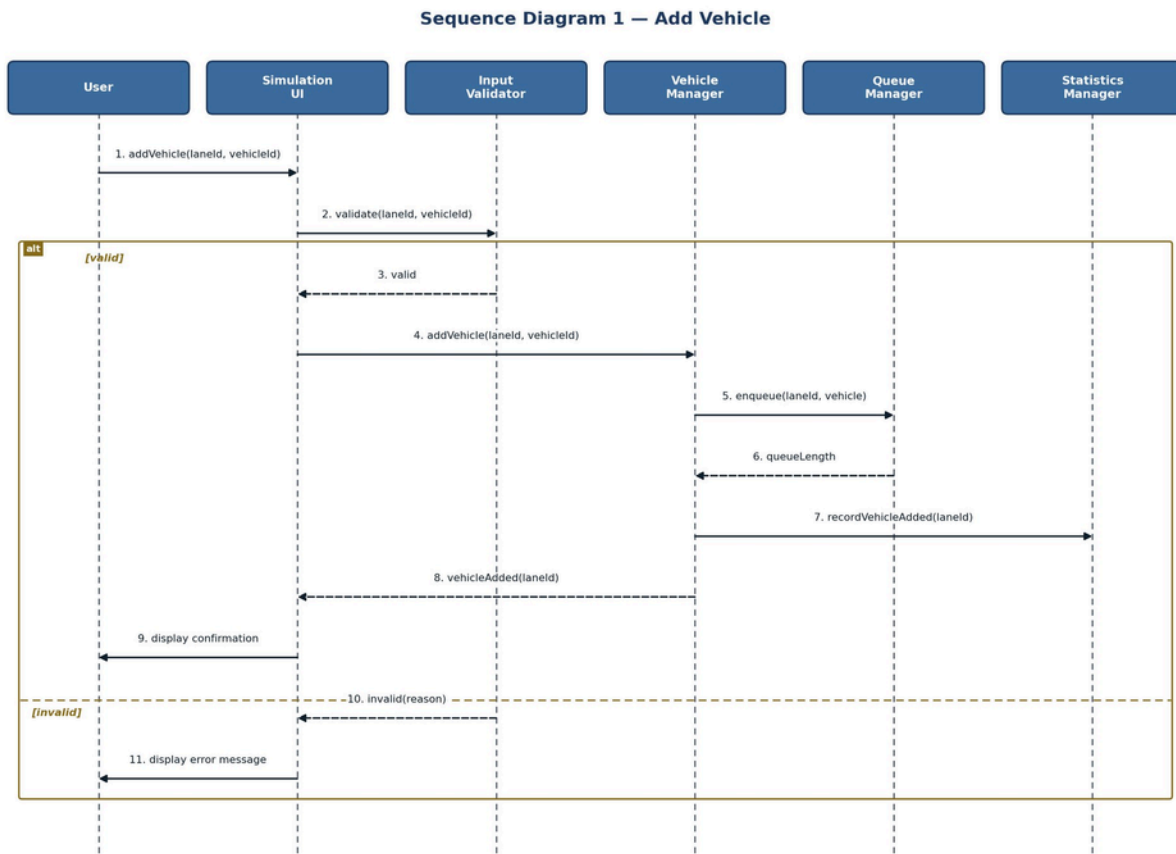


Figure 3 — Sequence diagram for the Add Vehicle flow (FR1, FR5, SR1).

### 8. Sequence Diagram — Vehicle Movement / Signal Change

Figure 4 shows what happens when the Signal Controller changes a lane's signal to Green. The Simulation Controller repeatedly asks the Queue Manager for the lane's front vehicle and directs the Vehicle Manager to move it out of the queue, for as long as the signal remains Green and the lane's queue is non-empty (the loop fragment). After the loop, the Simulation Controller checks for congestion and pulls updated statistics, then pushes the combined result to the Simulation UI for display — this is the flow that realises FR6 (display information) working together with FR3/FR4/FR5/FR7/FR8.

Sequence Diagram 2 — Vehicle Movement / Signal Change

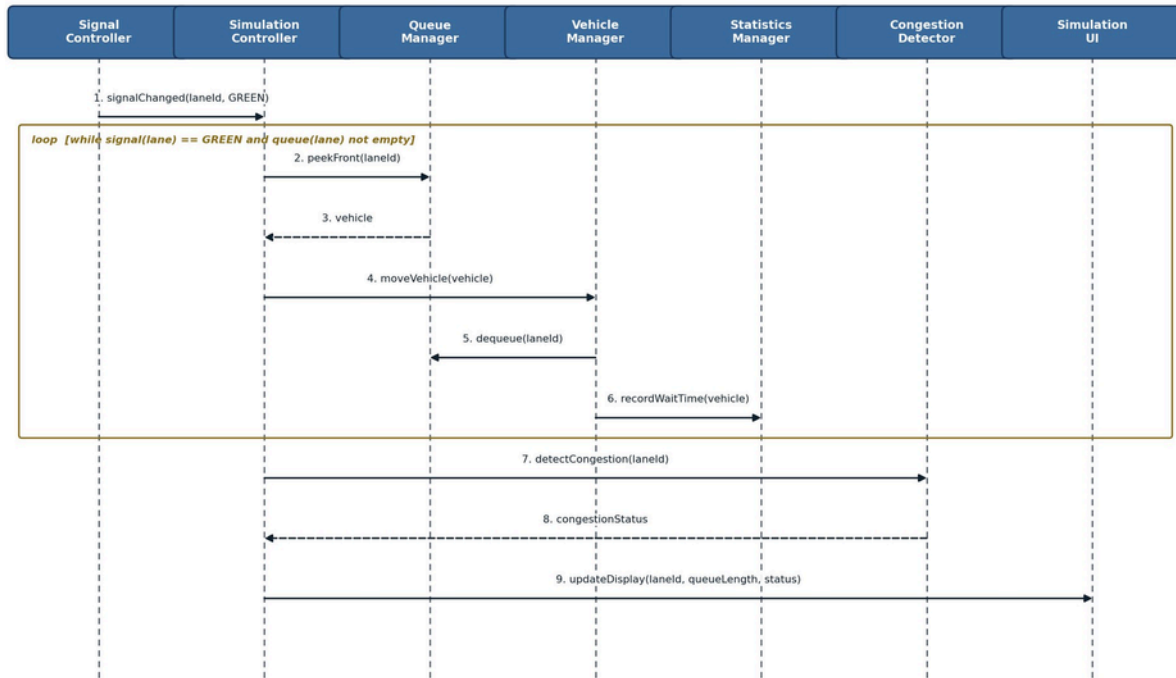


Figure 4 — Sequence diagram for vehicle movement triggered by a signal change (FR3, FR4, FR5, FR7, FR8).

## 9. API / Interface Design

Asthis is a C/C++ simulation rather than a networked service, its “API” is the set of function-level interfaces between components, not REST endpoints. The functions below are indicative of the design intent behind each component and are meant to guide implementation — the exact function names/signatures should be finalised once the team's actual code structure is settled, and this table updated to match.

| Function                      | Input                                                             | Output                                                      |
|-------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------|
| addVehicle(laneId, vehicleId) | Lane ID, vehicle ID                                               | Success/failure confirmation                                |
| removeVehicle(laneId)         | Lane ID                                                           | Removed vehicle, or an empty-queue error                    |
| getQueueLength(laneId)        | Lane ID                                                           | Integer queue length                                        |
| updateSignal()                | None (internal cycle/timer state)                                 | Updated signal state; triggers a signalChanged notification |
| getSignalState(laneId)        | Lane ID                                                           | Current signal state (Red / Yellow / Green)                 |
| detectCongestion(laneId)      | Lane ID                                                           | Congestion status (boolean / enum)                          |
| calculateStatistics()         | None (reads accumulated internal data)                            | Statistics summary: average wait                            |
| displayTrafficInfo()          | Aggregated display data (signals, queues, congestion, statistics) | time, queue length, vehicles processed                      |

Rendered console output

| Function            | Purpose                                                     | Provided by         | Consumed by                                            |
|---------------------|-------------------------------------------------------------|---------------------|--------------------------------------------------------|
| addVehicle          | Create and enqueue a new vehicle (FR1, FR5)                 | Vehicle Manager     | Simulation Controller                                  |
| removeVehicle       | Dequeue the front vehicle of a lane (FR5)                   | Queue Manager       | Vehicle Manager                                        |
| getQueueLength      | Report vehicles waiting in a lane (FR2, FR6, FR7)           | Queue Manager       | Congestion Detector, Statistics Manager, Simulation UI |
| updateSignal        | Advance each lane's signal per its cycle (FR3)              | Signal Controller   | Simulation Controller                                  |
| getSignalState      | Report a lane's current signal (FR4, FR6)                   | Signal Controller   | Simulation Controller, Simulation UI                   |
| detectCongestion    | Determine if a lane is congested (FR7)                      | Congestion Detector | Simulation Controller, Simulation UI                   |
| calculateStatistics | Compute wait time / queue length / vehicles processed (FR8) | Statistics Manager  | Simulation Controller, Simulation UI                   |
| displayTrafficInfo  | Present current traffic info to the user (FR6, NFR2)        | Simulation UI       | User                                                   |

## 10. Error Handling

Error handling follows a simple rule consistent with SR2 (queue data protection): an operation that cannot be completed safely is rejected before it touches shared state, rather than attempted and cleaned up afterwards.

| Error                                           | Detection Point                                     | System Response                                                                                         |
|-------------------------------------------------|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Invalid vehicle input                           | Input Validation (before Vehicle Manager is called) | Reject input and show an error message to the user.                                                     |
| Invalid lane ID                                 | Input Validation / Queue Manager                    | Reject the request; no queue is created or modified.                                                    |
| Empty queue (removeVehicle / dequeue attempted) | Queue Manager                                       | Do not attempt removal; return an empty-queue result instead of an error condition on the queue itself. |
| Invalid signal state                            | Signal Controller                                   | Reject the transition and hold the last known valid state; log/report the anomaly.                      |
| Invalid queue operation                         | Queue Manager                                       | Preserve queue integrity — the operation has no effect on stored vehicles (supports SR2 and NFR3).      |